

# Execution-Driven Congestion-Adaptive Guidance for Lifelong Multi-Robot Warehouse Navigation

Parth Oza  
Software Engineer  
Coordinatez Tec Inc.  
Chicago, IL, USA  
ozaparth055@gmail.com

Meet Patel  
Software Engineer  
Coordinatez Tec Inc.  
Chicago, IL, USA  
meet@coordinatez.com

**Abstract**—Most warehouse fleets that run hundreds of robots are coordinated by one-step planners in the PIBT family, because a one-step planner is fast and shrugs off execution errors. The price is myopia. Each robot descends a shortest-distance heuristic, so at high density everyone crowds into the same aisles, meets head-on, and throughput falls. The usual fix is a guidance graph, either optimised offline or learned, which needs a training phase per map and a model of the task mix. We wanted to know how far one can get with none of that, using only what a fleet manager already sees while the robots are running: who moved along which edge, and who had to wait. CA-PIBT keeps exponentially-decayed, direction-resolved traversal rates and a wait-heat map from executed motion, uses them as edge costs in per-robot distance tables that are refreshed on a staggered schedule, and adds a small stall-recovery rule. We test it on four public lifelong MAPF maps against PIBT, PIBT with hand-designed highways, and a windowed prioritised planner, over 1015 runs. The result depends on the map, and we say so. On the League of Robot Runners warehouse layout, CA-PIBT reaches 8.77 tasks per step with 400 robots, against 4.49 for highways and 2.91 for plain PIBT. On random-32-32-20, where highways have nothing to attach to, it more than doubles both. On the RHCR fulfilment layout and the sortation layout, whose wide corridors already suit static lanes, it ties with highways. Under stochastic execution delays it degrades far more slowly than the other PIBT variants, keeping 60% of its throughput at a 20% per-step delay probability where highway-PIBT keeps 23%. On a  $140 \times 500$  map with up to 4,000 robots, where density is low, the adaptive costs buy only 1–3% over highways and their per-step cost approaches one second on a single core, which we report as a limitation. The ablation shows that the congestion costs, not the recovery rule, carry the effect. Code, maps, raw logs and the script that produces every number here are released.

**Index Terms**—multi-agent path finding, lifelong planning, warehouse robotics, PIBT, traffic management, congestion

## I. INTRODUCTION

Automated fulfilment and sortation centres coordinate hundreds to thousands of mobile robots that continuously receive new pick-up and delivery goals. This is the *lifelong* multi-agent path finding (LMAPF) problem [1], [2]: unlike one-shot MAPF, there is no final configuration, so the objective is throughput, the number of goals reached per unit time, subject to a hard per-timestep planning budget.

Two families of solvers dominate practice. *Windowed search* (RHCR [2], windowed PIBT-LNS [3]) replans all agents every  $h$  steps with a bounded horizon; it produces high-quality paths

but its cost grows quickly with density and, being incomplete, it can livelock in corridors [4]. *One-step rule-based planners*, above all PIBT [5], [6], decide each step from the current configuration in microseconds per agent and are robust to execution errors because they never commit to a future. Almost every winning entry of the League of Robot Runners (LoRR) competition builds on PIBT [3], [7], [8].

PIBT’s limitation is that each agent greedily descends a distance-to-goal heuristic. At high density this is myopic: agents route through the same shortest corridors, meet head-on, and the priority-inheritance mechanism spends most of its effort untangling avoidable conflicts. The remedy of the last three years is *guidance*: bias each agent’s heuristic with traffic-aware edge costs. Guided-PIBT [9] plans time-independent guide paths that account for expected contra-flow; Guidance Graph Optimisation (GGO) [10] optimises a directed weighted graph offline with CMA-ES or a learned update network; Online GGO [11] learns a policy that updates edge weights from live traffic and reports up to +30.75% throughput over offline guidance. These methods are effective, but they require an offline optimisation or training phase for each map (or map family), a simulator of the task distribution, and in the online case a trained model.

The question we set out to answer is narrower and, we think, practical: how much of that benefit is available with no offline computation, no learning and no per-map tuning, from signals the fleet produces anyway while it runs? A fleet manager knows at every timestep which robot moved along which edge and which robot waited. It turns out that decayed statistics of just these two quantities, used as edge costs in per-robot distance tables, recover a large part of the gap on some layouts and none of it on others. Both outcomes are in the paper. Concretely:

- 1) We describe **CA-PIBT**, a congestion-adaptive PIBT whose guidance comes only from executed motion. Direction-resolved traversal rates penalise head-on traffic and a wait-heat map penalises hot spots. Tables are refreshed on a staggered schedule; the cost is 1–3 ms per step for 400 robots on small maps and grows to about a second per step for 4,000 robots on a  $140 \times 500$  map on one core.
- 2) We add a **stall-recovery rule** for robots that make

no progress for  $W$  steps, and evaluate every planner through the same **execution layer**, which resolves residual conflicts and can inject stochastic delays, so that the comparison is on equal footing.

- 3) We run a **controlled study**: 1015 runs, 5 seeds and 1000 steps each, against PIBT, PIBT with hand-designed highways [12] and windowed prioritised planning, on four public maps, plus ablations, a delay-robustness test and a scaling test to 4,000 robots. Where the method does not help, at low density and on layouts where static highways already do the job, we say that with the same prominence as where it does.

We make no claim against learned or offline-optimised guidance, because we did not run those systems. The claim is smaller: on layouts with narrow aisles or no lane structure, a large share of the benefit is available for free, and that matters when maps and task mixes change faster than anyone will retrain a model.

## II. RELATED WORK

**Lifelong MAPF and MAPD.** Ma et al. [1] formalised multi-agent pickup and delivery and introduced Token Passing, complete on well-formed instances; Liu et al. [13] added dummy paths for deadlock freedom when endpoints are known. RHCR [2] decomposes LMAPF into windowed MAPF instances replanned every  $h$  steps and scales to 1,000 agents; it is the standard search-based baseline. Windowed planning is incomplete, which motivated WinC-MAPF [4], DAG-ECBS [14] and Real-Time LaCAM [15], which restore completeness for one-shot MAPF. LaCAM and its successors [16]–[18] solve large one-shot instances in sub-second time; lifelong variants with local guidance appeared recently [19], [20]. Transient MAPF [21] shows that the requirement of simultaneous goal arrival in windowed replanning is itself a source of lost throughput.

**PIBT and the LoRR competition.** PIBT [5], [6] is a one-step planner with priority inheritance and backtracking, guaranteeing reachability on biconnected graphs. The LoRR competition [7], with orientation-aware robots, online task assignment and a 1 s/step budget at up to 10,000 agents, has been won in 2023 by windowed parallel PIBT-LNS [3] and in 2024 by multi-action PIBT [8]; both rely on guidance and on LNS improvement in the leftover budget. Jiang et al. [3] list congestion/myopia, the time budget and the model–reality gap as the field’s three open challenges; our method targets the first two with the third in mind.

**Guidance and traffic flow.** Highways [12] bias agents onto directional lanes by inflating the cost of moving against a lane direction; they are the classical hand-designed guidance and one of our baselines. Guided-PIBT [9] computes time-independent guide paths with congestion and contra-flow costs derived from other agents’ planned guide paths and uses them as PIBT heuristics. GGO [10] unifies highways and traffic rules as a directed weighted guidance graph optimised offline; Online GGO [11] learns an online update policy for the weights from live traffic statistics. Artificial potential fields

[22] and layout co-design [23], [24] are complementary levers. Our approach is closest in spirit to Online GGO’s hand-designed online baseline: the edge weights are an explicit function of observed traffic. The difference is that we derive the costs from *executed* motion with exponential forgetting, penalise head-on flow by direction, add wait heat, keep the tables per robot, and evaluate the design without any learned component.

**Learned solvers.** PRIMAL [25], PRIMAL2 [26], SCRIMP [27], Follower [28], MAPF-GPT [29], [30] and SILLM [31] learn decentralised policies, typically made collision-free by a PIBT wrapper [32]. Follower’s individual planner already uses a congestion-penalising heuristic, and SILLM uses dynamic guidance heuristics as network inputs; we isolate this ingredient and quantify its value without learning.

**Execution uncertainty.** MAPF with delay probabilities [33], kinematic lifelong planning [34], concurrent planning and execution under delays [35], and the SMART physics testbed [36] all show that plans rarely execute as scheduled. We therefore evaluate every planner through the same execution layer with configurable per-step delay probability.

## III. PROBLEM SETTING

We use a 4-connected grid  $G = (V, E)$  with obstacle cells removed.  $n$  robots occupy distinct vertices. At each discrete timestep  $t$  every robot either waits or moves to a neighbouring vertex; two robots may not occupy the same vertex at the same time nor traverse the same edge in opposite directions (vertex and swap conflicts). Following-moves into a vertex vacated at the same step are allowed, as in the LoRR rules. Each robot holds one goal; when it arrives, a new goal is sampled immediately. On LoRR maps goals alternate between a random *service* cell (shelf, ‘S’) and a random *emitter* cell (station, ‘E’); on the RHCR fulfilment map they alternate between endpoint and workstation cells; on random maps goals are uniform free cells. The primary metric is **throughput**, goals reached per timestep over a horizon of  $T$  steps. We also report the **wait fraction** (share of robot-steps spent stationary), **stall events** (a robot going  $W = 30$  consecutive steps without decreasing its heuristic distance to goal) and the **planning time per step**.

## IV. METHOD

### A. PIBT recap

PIBT [6] processes robots in decreasing priority; a robot sorts its candidate next vertices (four neighbours and its own cell) by a heuristic  $h_i(v)$  and takes the best vertex not already claimed, provided it does not swap with a neighbour. If the best vertex is occupied by a lower-priority robot  $j$  that has not yet decided,  $j$  inherits  $i$ ’s priority and is asked to move first; if  $j$  cannot,  $i$  backtracks to its next candidate. Priorities are the time elapsed since the robot’s current goal was assigned, with random tie breaking, so no robot starves. The heuristic is normally the true grid distance to the goal, obtained by a breadth-first search from the goal and stored per robot.

### B. Execution-driven congestion statistics

Let  $f_t(v, d) \in \mathbb{R}_{\geq 0}$  be the decayed count of robots that entered vertex  $v$  moving in direction  $d \in \{E, S, W, N\}$ , and  $w_t(v)$  the decayed count of wait actions executed at  $v$ :

$$f_{t+1}(v, d) = \lambda f_t(v, d) + \mathbb{1}[\text{some robot moved into } v \text{ along } d \text{ at } t], \quad (1)$$

$$w_{t+1}(v) = \lambda w_t(v) + \mathbb{1}[\text{some robot waited at } v \text{ at } t]. \quad (2)$$

With  $\lambda = 0.97$  the memory is  $\approx 33$  steps, long enough to average out individual robots and short enough to follow shifts in the task mix. Multiplying by  $(1 - \lambda)$  turns the counters into rates in  $[0, 1]$ : the fraction of recent timesteps in which the event occurred.

### C. Congestion-adaptive guidance tables

For a robot heading to goal  $g$  we compute a distance table  $h(\cdot)$  by Dijkstra from  $g$  over reversed edges, where the cost of traversing edge  $u \rightarrow v$  in direction  $d$  is

$$c(u \rightarrow v, d) = b(u \rightarrow v, d) + \alpha(1 - \lambda)f_t(v, \bar{d}) + \gamma(1 - \lambda)w_t(v), \quad (3)$$

where  $\bar{d}$  is the direction opposite to  $d$ , so the second term charges for entering a cell that recent traffic has been entering from the opposite side (head-on flow), and the third term charges for entering a cell where robots have recently been forced to wait. The base cost  $b$  is either 1 or a static highway cost [12] that adds a penalty  $\rho = 1$  for moving against the lane direction of the current row or column (rows alternate east/west, columns alternate south/north). The static highway is a prior; the two adaptive terms correct it where actual traffic disagrees with it. We use  $\alpha = 10$ ,  $\gamma = 5$  for every map and fleet size; these were chosen on two maps at one density (Section V) and never tuned afterwards. Costs are stored as 16-bit fixed-point numbers with a scale of 4.

A table is rebuilt when a robot receives a new goal and thereafter every  $K = 20$  steps, staggered so that on each timestep only robots with  $(t + i) \bmod K = 0$  refresh. A Dijkstra over the  $33 \times 57$  LoRR warehouse costs about 0.1 ms and over the  $140 \times 500$  warehouse about 3 ms, so with  $n$  robots the amortised guidance cost is  $\approx n/K$  Dijkstra runs per step plus one per finished task. The current-step decision is otherwise exactly PIBT with  $h_i$  replaced by the weighted table, so the completeness argument of PIBT for reaching a fixed goal is unaffected (the table is a valid finite potential on the connected component).

### D. Stall recovery

A robot tracks the best (lowest) heuristic value it has reached since its goal was assigned. If this value has not improved for  $W = 30$  steps, the robot is *stalled*. In all PIBT variants a stalled robot receives a large priority boost (the standard elapsed-time priority already grows, but slowly). CA-PIBT additionally assigns a temporary *retreat goal*: the unoccupied cell within 3 steps with the lowest wait heat, pursued for 6 steps, after which the robot's table is rebuilt with the current traffic statistics and it resumes its real goal. The intent is to

break symmetric stand-offs in narrow aisles by pulling one participant into a quiet pocket. As Section VI-B shows, this mechanism is cheap but contributes little once congestion-adaptive costs are active; we report it because it is a natural first idea and its (small) effect should be documented.

### E. Execution layer and delays

All planners emit a desired next vertex per robot. The execution layer (i) with probability  $p$  per robot and step replaces a move by a wait (a delayed robot), (ii) resolves vertex conflicts in favour of the robot that stays or, between two movers, the higher-priority robot, (iii) blocks a robot from entering a cell whose occupant is not moving, and (iv) cancels both moves of a swap. It iterates until no change. For PIBT the layer is idle when  $p = 0$  (PIBT is conflict-free by construction); for the windowed planner it also catches the residual conflicts created when a robot fails to find a path within the expansion limit. Statistics for Eq. (3) are taken from the moves that were actually executed.

## V. EXPERIMENTAL SETUP

**Maps.** We use four public layouts: `warehouse_small` ( $33 \times 57$ , 1,277 free cells, 40 emitters, 342 service cells) and `sortation_small` ( $33 \times 57$ , 1,564 free cells) from the LoRR 2023/2024 benchmark archive [7]; the fulfilment map `kiva` ( $33 \times 46$ , 1,278 free cells) distributed with RHCR [2]; and `random-32-32-20` (819 free cells) from the MovingAI MAPF benchmark [37]. For the scaling study we use LoRR `warehouse_large` ( $140 \times 500$ , 38,586 free cells). Fleet sizes range from 50 to 400 robots on the warehouse maps (up to 31% of free cells), 50–600 on sortation, 50–300 on random, and 1,000–4,000 on the large warehouse.

**Planners.** *PIBT*: the baseline with BFS distance tables. *H-PIBT*: PIBT with static highway tables ( $\rho = 1$ ), representing hand-designed guidance [10], [12]. *WPP*: windowed prioritised planning in the RHCR style [2], [38]: every  $h = 10$  steps all robots are planned sequentially in priority order with space-time  $A^*$  over a horizon of 20 steps (expansion limit 20,000); robots that cannot be planned hold position and the remaining robots are replanned around them for up to three rounds; plans are kept across goal changes until the next window. WPP is a simplified stand-in for RHCR, which uses PBS or ECBS as the windowed solver; we could not run the original implementation in our environment and we therefore make no claim about RHCR itself. *CA-PIBT*: the proposed method with highway base costs,  $\alpha = 10$ ,  $\gamma = 5$ ,  $\lambda = 0.97$ ,  $K = 20$ ,  $W = 30$ . Ablations: *CA-PIBT-noRecover* (no retreat goal), *CA-PIBT-noHighway* (uniform base cost  $b = 1$ ), and *H-PIBT+Recover* ( $\alpha = \gamma = 0$ , isolating stall recovery). All PIBT variants share the same stall priority boost so that the comparison isolates guidance.

**Protocol.** Every configuration is run for  $T = 1000$  steps with 5 seeds (3 on the large map); seeds fix initial positions and goal sequences. Hyper-parameters  $\alpha, \gamma$  were selected once from the grid  $\alpha \in \{5, 10, 20, 40\}$ ,  $\gamma \in \{0, 5, 15\}$  on `warehouse_small` and `kiva` with 250 robots and 500

TABLE I  
THROUGHPUT (TASKS/STEP, MEAN $\pm$ 95% CI) / PLANNING TIME  
(MS/STEP) AT A MODERATE AND AT THE LARGEST EVALUATED FLEET ON  
EACH MAP.

| Map                              | $n$ | PIBT                   | H-PIBT                  | WPP                     | CA-PIBT (ours)          |
|----------------------------------|-----|------------------------|-------------------------|-------------------------|-------------------------|
| warehouse_small (33 $\times$ 57) | 250 | 3.64 $\pm$ 0.16 / 0.07 | 5.58 $\pm$ 0.03 / 0.08  | 1.54 $\pm$ 0.64 / 29.63 | 6.52 $\pm$ 0.03 / 1.31  |
| warehouse_small (33 $\times$ 57) | 400 | 2.91 $\pm$ 0.06 / 0.12 | 4.49 $\pm$ 0.15 / 0.13  | 0.90 $\pm$ 0.10 / 55.28 | 8.77 $\pm$ 0.06 / 2.49  |
| kiva (33 $\times$ 46)            | 250 | 5.96 $\pm$ 0.04 / 0.07 | 7.49 $\pm$ 0.03 / 0.08  | 6.34 $\pm$ 0.20 / 2.10  | 7.55 $\pm$ 0.04 / 1.18  |
| kiva (33 $\times$ 46)            | 400 | 7.33 $\pm$ 0.19 / 0.12 | 10.46 $\pm$ 0.04 / 0.14 | 6.43 $\pm$ 2.80 / 35.09 | 10.95 $\pm$ 0.09 / 2.08 |
| sortation_small (33 $\times$ 57) | 300 | 6.62 $\pm$ 0.02 / 0.08 | 7.61 $\pm$ 0.03 / 0.10  | 2.97 $\pm$ 0.58 / 52.38 | 7.74 $\pm$ 0.04 / 1.84  |
| sortation_small (33 $\times$ 57) | 600 | 6.87 $\pm$ 0.17 / 0.18 | 12.02 $\pm$ 0.05 / 0.21 | 1.78 $\pm$ 0.22 / 78.95 | 12.12 $\pm$ 0.09 / 4.52 |
| random-32-32-20                  | 200 | 2.23 $\pm$ 1.23 / 0.06 | 2.65 $\pm$ 0.73 / 0.06  | 2.76 $\pm$ 0.48 / 20.76 | 6.70 $\pm$ 0.07 / 0.59  |
| random-32-32-20                  | 300 | 3.05 $\pm$ 1.04 / 0.09 | 3.39 $\pm$ 0.63 / 0.10  | 0.91 $\pm$ 0.07 / 37.89 | 7.85 $\pm$ 0.19 / 1.11  |

steps (2 seeds), and then frozen; the grid results are included in the released data. Error bars are 95% confidence intervals over seeds. All experiments ran single-threaded on a 2-vCPU cloud container (g++ 13, -O2); absolute times are indicative only. The simulator, maps, raw CSV logs and the analysis script that generates every table, figure and number in this paper are released with the paper.

## VI. RESULTS

### A. Throughput versus fleet size

Figure 1 and Table I carry the main result. Below roughly 100 robots the PIBT variants are indistinguishable; there is no congestion for the guidance to react to. As the fleet grows, plain PIBT saturates and then falls (on warehouse\_small it peaks near 200 robots), because the one-cell aisles between shelf blocks fill with head-on encounters. Highways push the collapse out. CA-PIBT does not collapse within the range we tested: with 400 robots on warehouse\_small it reaches 8.77 tasks per step, against 4.49 for H-PIBT and 2.91 for PIBT, a gain of 95% and 201% respectively. On random-32-32-20 the difference is the largest we saw. There is no lane structure for highways to attach to, PIBT and H-PIBT swing wildly from seed to seed (some seeds lock up in the narrow passages, visible as the wide error bars), and CA-PIBT more than doubles their throughput with tight intervals. On kiva and sortation\_small the story is different. The alternating-direction highway prior already removes most head-on traffic in their wide, regular corridors, and CA-PIBT ties with H-PIBT to within a few percent (Table I). So the adaptive terms earn their keep where the static prior is wrong or has nothing to grip, and are close to neutral where it is right. WPP is competitive with PIBT at low density but its throughput falls with density as robots that cannot be planned within the window hold position and block others, reproducing the known incompleteness of windowed replanning [4]; its per-step cost also grows by two orders of magnitude (Table I).

Figure 2 explains the mechanism: CA-PIBT roughly halves the wait fraction relative to H-PIBT at high density and removes almost all stall events, whereas PIBT accumulates hundreds of stalls per 1,000 steps.

### B. Ablation

Table II and Fig. 3 take the method apart at the largest fleet on two maps. Three things stand out. First, once the adaptive costs are on, the retreat rule changes nothing: CA-PIBT

TABLE II  
ABLATION AT THE LARGEST EVALUATED FLEET.

| Map             | $n$ | Variant           | Throughput       | Wait frac. | Stalls | ms/step |
|-----------------|-----|-------------------|------------------|------------|--------|---------|
| warehouse_small | 400 | H-PIBT            | 4.49 $\pm$ 0.15  | 0.289      | 2231.2 | 0.13    |
| warehouse_small | 400 | H-PIBT+Recover    | 5.56 $\pm$ 0.14  | 0.247      | 1271.2 | 2.80    |
| warehouse_small | 400 | CA-PIBT-noHighway | 8.44 $\pm$ 0.10  | 0.131      | 335.8  | 1.96    |
| warehouse_small | 400 | CA-PIBT-noRecover | 8.74 $\pm$ 0.14  | 0.130      | 145.2  | 2.77    |
| warehouse_small | 400 | CA-PIBT           | 8.77 $\pm$ 0.06  | 0.131      | 137.6  | 2.49    |
| kiva            | 400 | H-PIBT            | 10.46 $\pm$ 0.04 | 0.088      | 26.6   | 0.14    |
| kiva            | 400 | H-PIBT+Recover    | 10.40 $\pm$ 0.06 | 0.088      | 23.0   | 1.91    |
| kiva            | 400 | CA-PIBT-noHighway | 10.70 $\pm$ 0.06 | 0.077      | 65.4   | 1.74    |
| kiva            | 400 | CA-PIBT-noRecover | 10.98 $\pm$ 0.07 | 0.083      | 19.6   | 2.08    |
| kiva            | 400 | CA-PIBT           | 10.95 $\pm$ 0.09 | 0.085      | 19.2   | 2.08    |

TABLE III  
THROUGHPUT WITH 300 ROBOTS UNDER EXECUTION DELAYS.

| Map ( $n=300$ ) | Planner | $p=0$ | $p=0.05$ | $p=0.1$ | $p=0.2$ |
|-----------------|---------|-------|----------|---------|---------|
| warehouse_small | PIBT    | 3.20  | 1.91     | 1.38    | 0.86    |
| warehouse_small | H-PIBT  | 5.48  | 3.70     | 2.31    | 1.29    |
| warehouse_small | WPP     | 0.99  | 0.99     | 0.92    | 0.98    |
| warehouse_small | CA-PIBT | 7.53  | 6.73     | 5.86    | 4.51    |
| kiva            | PIBT    | 6.66  | 5.04     | 3.03    | 1.47    |
| kiva            | H-PIBT  | 8.74  | 7.62     | 6.54    | 3.57    |
| kiva            | WPP     | 6.81  | 7.61     | 6.76    | 4.88    |
| kiva            | CA-PIBT | 8.84  | 8.09     | 7.30    | 5.89    |

and CA-PIBT-noRecover differ by less than their confidence intervals on both maps. Second, the retreat rule is not useless in general. Added to plain highways on warehouse\_small (H-PIBT+Recover) it lifts throughput from 4.49 to 5.56 and cuts stall events roughly in half, because at 400 robots the static lanes produce many symmetric stand-offs that a short retreat breaks. It simply becomes redundant when traffic is separated by direction before the stand-offs form. On kiva it does nothing either way. Third, the highway prior is worth a little but not much: dropping it (CA-PIBT-noHighway) costs 8.44 versus 8.77 on warehouse\_small and 10.70 versus 10.95 on kiva. The execution-driven congestion costs are where the throughput comes from.

### C. Robustness to execution delays

Real robots slip, wait for sensors and yield to people. Fig. 4 and Table III apply a per-robot, per-step delay probability  $p \in \{0.05, 0.1, 0.2\}$  with 300 robots. Every planner loses throughput, but not equally. On warehouse\_small at  $p = 0.2$ , PIBT keeps 27% of its delay-free throughput and H-PIBT keeps 23%, while CA-PIBT keeps 60%; on kiva the figures are 22%, 41% and 67%. We think the reason is that a delayed robot is, from the point of view of the statistics, a robot that waited, so the delay shows up as heat and the fleet routes around the affected cells within a few dozen steps. WPP's retention looks high on warehouse\_small only because it had already collapsed at  $p = 0$ ; on kiva, where it was functioning, it also holds up reasonably, since a delayed robot only invalidates reservations until the next window.

### D. Scaling to 4,000 robots

The 140  $\times$  500 LoRR warehouse (Fig. 5, Table IV) is a different regime. Even 4,000 robots occupy only about 10%

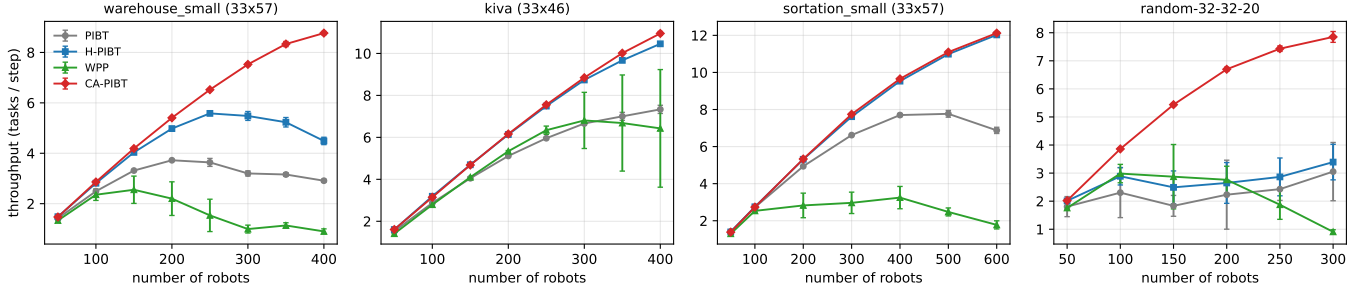


Fig. 1. Throughput versus number of robots on four maps (mean and 95% CI over 5 seeds, 1000 steps). CA-PIBT is the proposed method; WPP is windowed prioritised planning ( $h = 10$ , horizon 20).

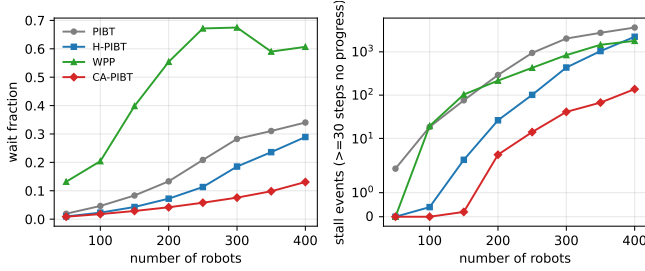


Fig. 2. Wait fraction (left) and stall events (right, symlog) on warehouse\_small. CA-PIBT lowers both by routing around head-on traffic before it forms.

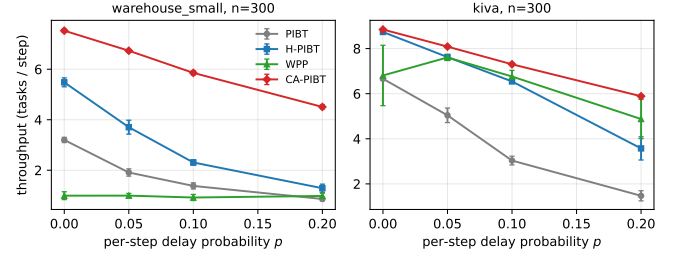


Fig. 4. Throughput under per-step execution delay probability  $p$  with 300 robots.

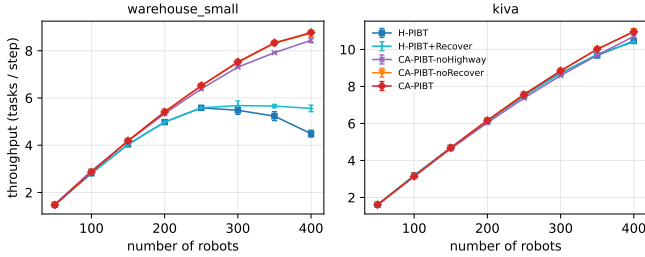


Fig. 3. Ablation: contribution of the highway prior, the adaptive costs and stall recovery.

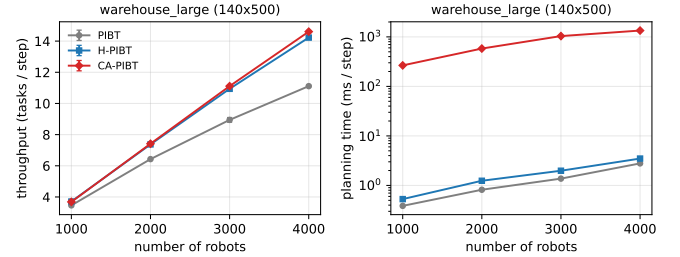


Fig. 5. LoRR warehouse\_large ( $140 \times 500$ ): throughput and planning time per step (log scale) for 1,000–4,000 robots.

## VII. LIMITATIONS

of its free cells, aisles are long and traffic rarely opposes itself, so there is little for the adaptive costs to fix: CA-PIBT beats PIBT by 31% at 4,000 robots but beats H-PIBT by only 3%, and at 1,000 robots the two are tied. Planning time is the other side of this coin. A Dijkstra over 38,586 cells costs a few milliseconds, and with thousands of robots finishing tasks and refreshing tables the total reaches 1038 ms per step at 3,000 robots and 1342 ms at 4,000, measured while two runs shared a 2-vCPU container. That is at or above the LoRR budget of one second, whereas H-PIBT needs 3 ms. On large, sparse maps the honest recommendation is therefore static highways, or CA-PIBT with a longer refresh interval and incremental table updates, which we have not implemented. We did not run WPP at this scale.

(i) We did not compare against Guided-PIBT, GGO or Online GGO, nor against the original RHCR code; the WPP baseline is our own simplified windowed planner. Our claims are therefore relative to PIBT, highways and windowed PP only. (ii) Robots are holonomic unit-speed point agents without rotation; LoRR’s orientation model and real kinodynamics [34], [36] are not modelled. (iii) Task sampling is uniform over service and emitter cells; skewed or time-varying demand may favour or hurt exponential forgetting, and  $\lambda$  was not tuned. (iv) The maps are small to medium; we ran three seeds at 4,000 robots. (v) The gains vanish at low density, are within a few percent of static highways on kiva, sortation\_small and the large warehouse, and the per-robot Dijkstra tables become the bottleneck beyond a few thousand robots on large maps. The method pays off on dense layouts with narrow aisles

TABLE IV  
WAREHOUSE\_LARGE: THROUGHPUT (MEAN $\pm$ 95% CI OVER 3 SEEDS) /  
MS PER STEP.

| $n$  | PIBT               | H-PIBT             | CA-PIBT (ours)        |
|------|--------------------|--------------------|-----------------------|
| 1000 | 3.5 $\pm$ 0.0 / 0  | 3.7 $\pm$ 0.0 / 1  | 3.7 $\pm$ 0.1 / 266   |
| 2000 | 6.4 $\pm$ 0.0 / 1  | 7.4 $\pm$ 0.0 / 1  | 7.4 $\pm$ 0.0 / 581   |
| 3000 | 8.9 $\pm$ 0.2 / 1  | 10.9 $\pm$ 0.1 / 2 | 11.1 $\pm$ 0.1 / 1038 |
| 4000 | 11.1 $\pm$ 0.1 / 3 | 14.2 $\pm$ 0.0 / 3 | 14.6 $\pm$ 0.1 / 1342 |

or no lane structure; it is not a universal improvement.

## VIII. CONCLUSION

A one-step planner can be made a good deal less myopic without learning or offline optimisation. Decayed, direction-resolved traversal rates and wait heat, measured from what the robots actually did and used as edge costs in per-robot distance tables, raised PIBT’s throughput by large factors on a dense warehouse layout and on an unstructured map, kept it working where windowed prioritised planning collapsed, and made it markedly more tolerant of execution delays, all for a few milliseconds per step on small maps. The same statistics did nothing on layouts where static lanes already separate traffic, and their per-step cost grows uncomfortably on very large maps. The obvious next steps are to use an offline-optimised guidance graph as the base cost and let the execution statistics correct it online, to make the table updates incremental, to add orientation and kinodynamic constraints, and to try the idea on a physics testbed.

## REPRODUCIBILITY

Everything needed to regenerate this paper is public at <https://github.com/Parth-Oza/ca-pibt>: the single-file C++17 simulator containing all four planners, the maps, the job list for all 1015 runs, the raw per-run logs, and `analyze.py`, which reproduces every figure, table and inline number here from those logs. Reproducing the small-map experiments takes a few CPU-hours and the large-map runs a few hours more, single-threaded.

## REFERENCES

- [1] H. Ma, J. Li, T. K. S. Kumar, and S. Koenig, “Lifelong multi-agent path finding for online pickup and delivery tasks,” in *Proceedings of the 16th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2017, pp. 837–845, arXiv:1705.10868.
- [2] J. Li, A. Tinka, S. Kiesel, J. W. Durham, T. K. S. Kumar, and S. Koenig, “Lifelong multi-agent path finding in large-scale warehouses,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, arXiv:2005.07371.
- [3] H. Jiang, Y. Zhang, R. Veerapaneni, and J. Li, “Scaling lifelong multi-agent path finding to more realistic settings: Research challenges and opportunities,” in *Proceedings of the International Symposium on Combinatorial Search (SoCS)*, vol. 17, 2024.
- [4] R. Veerapaneni, M. S. Saleem, J. Li, and M. Likhachev, “Windowed MAPF with completeness guarantees,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025, arXiv:2410.01798.
- [5] K. Okumura, M. Machida, X. Défago, and Y. Tamura, “Priority inheritance with backtracking for iterative multi-agent path finding,” in *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 2019, pp. 535–542.
- [6] —, “Priority inheritance with backtracking for iterative multi-agent path finding,” *Artificial Intelligence*, vol. 310, p. 103752, 2022.
- [7] S.-H. Chan, Z. Chen, T. Guo, H. Zhang, Y. Zhang, D. Harabor, S. Koenig, C. Wu, and J. Yu, “The league of robot runners: Competition goals, designs, and implementation,” in *ICAPS 2024 System Demonstration Track*, 2024. [Online]. Available: <https://harabor.net/data/papers/lorr-demo-icaps24.pdf>
- [8] E. Yuhnevich and A. Andreychuk, “Enhancing PIBT via multi-action operations,” *arXiv preprint arXiv:2511.09193*, 2025.
- [9] Z. Chen, D. Harabor, J. Li, and P. J. Stuckey, “Traffic flow optimisation for lifelong multi-agent path finding,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024.
- [10] Y. Zhang, H. Jiang, V. Bhatt, S. Nikolaidis, and J. Li, “Guidance graph optimization for lifelong multi-agent path finding,” in *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.
- [11] H. Zang, Y. Zhang, H. Jiang, Z. Chen, D. Harabor, P. J. Stuckey, and J. Li, “Online guidance graph optimization for lifelong multi-agent path finding,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025.
- [12] L. Cohen, T. Uras, and S. Koenig, “Feasibility study: Using highways for bounded-suboptimal multi-agent path finding,” in *Proceedings of the International Symposium on Combinatorial Search (SoCS)*, 2015.
- [13] M. Liu, H. Ma, J. Li, and S. Koenig, “Task and path planning for multi-agent pickup and delivery,” in *Proceedings of the 18th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2019, pp. 1152–1160.
- [14] T. Zhang, R. Veerapaneni, S.-H. Chan, J. Li, and M. Likhachev, “Dynamic agent grouping ECBS: Scaling windowed multi-agent path finding with completeness guarantees,” *arXiv preprint arXiv:2509.15381*, 2025.
- [15] R. Liang, R. Veerapaneni, D. Harabor, J. Li, and M. Likhachev, “Real-time LaCAM for real-time MAPF,” in *Proceedings of the International Symposium on Combinatorial Search (SoCS)*, vol. 18, 2025.
- [16] K. Okumura, “LaCAM: Search-based algorithm for quick multi-agent pathfinding,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, 2023, arXiv:2211.13432.
- [17] —, “Improving LaCAM for scalable eventually optimal multi-agent pathfinding,” in *Proceedings of the 32nd International Joint Conference on Artificial Intelligence (IJCAI)*, 2023, pp. 243–251.
- [18] —, “Engineering LaCAM\*: Towards real-time, large-scale, and near-optimal multi-agent pathfinding,” in *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2024, pp. 1501–1509, arXiv:2308.04292.
- [19] T. Arita and K. Okumura, “Local guidance for configuration-based multi-agent pathfinding,” *arXiv preprint arXiv:2510.19072*, 2025.
- [20] —, “Lifelong LaCAM with local guidance for lifelong MAPF,” in *Proceedings of the International Symposium on Combinatorial Search (SoCS)*, 2026, arXiv:2605.16855.
- [21] J. Morag, N. Gabay, D. Koyfman, and R. Stern, “Transient multi-agent path finding for lifelong navigation in dense environments,” *arXiv preprint arXiv:2412.04256*, 2024.
- [22] A. Pertzovsky, R. Stern, A. Felner, and R. Zivan, “Enhancing lifelong multi-agent path-finding by using artificial potential fields,” *arXiv preprint arXiv:2505.22753*, 2025.
- [23] Y. Zhang, M. C. Fontaine, V. Bhatt, S. Nikolaidis, and J. Li, “Multi-robot coordination and layout design for automated warehousing,” in *Proceedings of the 32nd International Joint Conference on Artificial Intelligence (IJCAI)*, 2023.
- [24] —, “Arbitrarily scalable environment generators via neural cellular automata,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2310.18622.
- [25] G. Sartoretti, J. Kerr, Y. Shi, G. Wagner, T. K. S. Kumar, S. Koenig, and H. Choset, “PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning,” *IEEE Robotics and Automation Letters*, vol. 4, no. 3, pp. 2378–2385, 2019.
- [26] M. Damani, Z. Luo, E. Wenzel, and G. Sartoretti, “PRIMAL<sub>2</sub>: Pathfinding via reinforcement and imitation multi-agent learning – lifelong,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 2666–2673, 2021, arXiv:2010.08184.
- [27] Y. Wang, B. Xiang, S. Huang, and G. Sartoretti, “SCRIMP: Scalable communication for reinforcement- and imitation-learning-based multi-agent pathfinding,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.

- [28] A. Skrynnik, A. Andreychuk, M. Nesterova, K. Yakovlev, and A. Panov, "Learn to follow: Decentralized lifelong multi-agent pathfinding via planning and learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024.
- [29] A. Andreychuk, K. Yakovlev, A. Panov, and A. Skrynnik, "MAPF-GPT: Imitation learning for multi-agent pathfinding at scale," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025.
- [30] A. Andreychuk, K. Yakovlev, A. I. Panov, and A. Skrynnik, "Advancing learnable multi-agent pathfinding solvers with active fine-tuning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025.
- [31] H. Jiang, Y. Wang, R. Veerapaneni, T. H. Duhan, G. A. Sartoretti, and J. Li, "Deploying ten thousand robots: Scalable imitation learning for lifelong multi-agent path finding," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, arXiv:2410.21415.
- [32] R. Veerapaneni, Q. Wang, K. Ren, A. Jakobsson, J. Li, and M. Likhachev, "Improving learnt local MAPF policies with heuristic search," in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2024, arXiv:2403.20300.
- [33] H. Ma, T. K. S. Kumar, and S. Koenig, "Multi-agent path finding with delay probabilities," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2017, arXiv:1612.05309.
- [34] H. Ma, W. Hönig, T. K. S. Kumar, N. Ayanian, and S. Koenig, "Lifelong path planning with kinematic constraints for multi-agent pickup and delivery," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 7651–7658.
- [35] Y. Zhang, Z. Chen, D. Harabor, P. L. Bodic, and P. J. Stuckey, "Concurrent planning and execution in lifelong multi-agent path finding with delay probabilities," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025.
- [36] J. Yan, Z. Li, W. Kang, Y. Zhang, S. Smith, and J. Li, "Advancing MAPF towards the real world: A scalable multi-agent realistic testbed (SMART)," in *ICAPS 2025 System Demonstration Track*, 2025, arXiv:2503.04798.
- [37] R. Stern, N. R. Sturtevant, A. Felner, S. Koenig, H. Ma, T. T. Walker, J. Li, D. Atzmon, L. Cohen, T. K. S. Kumar, E. Boyarski, and R. Barták, "Multi-agent pathfinding: Definitions, variants, and benchmarks," in *Proceedings of the International Symposium on Combinatorial Search (SoCS)*, 2019, arXiv:1906.08291; defines the MovingAI MAPF benchmark.
- [38] D. Silver, "Cooperative pathfinding," in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2005, pp. 117–122.